অধ্যায় ২৩

২৩

JavaScript Performance Optimization, Memory Management, Profiling এবং Web Application Performance

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- অনুমানের বদলে Profiling দিয়ে খুঁজে বের করতে, সময় আসলে কোথায় যাচ্ছে।
- User Timing API (`performance.mark` , `performance.measure`) ও `PerformanceObserver` দিয়ে নিজের Code-এ মাপার বিন্দু বসাতে।
- Node.js-এর Inspector দিয়ে CPU Profile নিতে এবং তার Self Time পড়তে।
- Event Loop Delay মাপতে, এবং Heap-এর বৃদ্ধি দেখে Memory Leak শনাক্ত করতে।
- Browser-এর Rendering Pipeline (Style, Layout, Paint, Composite) বুঝে কোন CSS পরিবর্তন সস্তা, কোনটি দামি তা বলতে।
- Virtual Scrolling দিয়ে হাজারো সারির তালিকা অল্প DOM-এ দেখাতে।
- Web Worker-এ ভারী কাজ সরিয়ে Main Thread মুক্ত রাখতে, এবং Transferable Object-এর সুবিধা বুঝতে।
- একটি সম্পূর্ণ **TravelSearch Explorer** তৈরি করতে: ১২,০০০ Record, Index-ভিত্তিক খোঁজ, Worker ও Virtual List।

পূর্বশর্ত

- অধ্যায় ১৯: পরিমাপের পদ্ধতি (Warm-up, Median), Big-O, Long Task, Layout Thrashing, Garbage Collection, `WeakMap`।
- অধ্যায় ১০ ও ১৪: Event Loop, Debounce।
- অধ্যায় ১২: ES Module।
- অধ্যায় ২২: Property-based Test, Browser-এ E2E।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১৯ একটি নীতি দিয়ে শেষ হয়েছিল: আগে মাপুন, তারপর Optimize করুন। কিন্তু একটি বাস্তব Application-এ শত শত Function, এবং "ধীর" অভিযোগটি প্রায়ই অস্পষ্ট: "খোঁজার সময় Page আটকে যায়"। কোন Function দায়ী? Rendering, না হিসাব? Memory কি ধীরে ধীরে বাড়ছে? এই প্রশ্নের উত্তর অনুমানে দেওয়া যায় না; দরকার যন্ত্র—**Profiler**—যা Program চলার সময় দেখে আসলে কী ঘটছে।

এই অধ্যায়ের দ্বিতীয় অংশ Browser-এর বিশেষ সমস্যা: হাজারো সারির তালিকা, এবং ভারী হিসাব যা Main Thread আটকে রাখে। দুটি কৌশল—Virtual Scrolling ও Web Worker—আধুনিক Web Application-এর অপরিহার্য হাতিয়ার, এবং দুটিই ভুলভাবে প্রয়োগ করা সহজ।

২৩.১ আগে যা শিখেছেন, এখন যা নতুন

v30-এর অধ্যায় ১৯ ও ২৩—দুটিই Performance নিয়ে। পুনরাবৃত্তি এড়াতে এই অধ্যায় অধ্যায় ১৯-এর ধারণাগুলো সংক্ষেপে উল্লেখ করে এবং নতুন স্তরে যায়:

অধ্যায় ১৯-এ শেখা	এই অধ্যায়ে নতুন
`performance.now()`, Warm-up, Median	User Timing (`mark`, `measure`), `PerformanceObserver`
DevTools Performance Panel-এর ধারণা	Programmatic CPU Profile, Self Time-এর বিশ্লেষণ
Long Task, কাজ টুকরোয় ভাগ	Event Loop Delay-এর Histogram
GC ও Memory Leak-এর ধারণা	Heap মেপে Leak প্রমাণ
Layout Thrashing	Rendering Pipeline, `transform` বনাম Layout
বড় তালিকায় Virtualization-এর উল্লেখ	Virtual Scrolling-এর পূর্ণ Implementation ও Test
Web Worker-এর উল্লেখ	Module Worker, Message, Transferable

২৩.২ মাপার বিন্দু: User Timing API

`performance.now()` দিয়ে শুরু ও শেষ বিয়োগ করা কাজ করে, কিন্তু ফলটি শুধু একটি Variable-এ থাকে। **User Timing API** মাপকে নামযুক্ত Entry বানায়: `performance.mark(name)` একটি মুহূর্ত চিহ্নিত করে, `performance.measure(name, startMark, endMark)` দুটির মধ্যের সময় একটি Entry হিসেবে রাখে। Browser-এর DevTools এই Entry-গুলো Performance Panel-এর "Timings" সারিতে দেখায়, তাই আপনার নিজের কাজ Profile-এর ভেতরে নাম ধরে খুঁজে পাওয়া যায়।

user-timing.js · JavaScript — চলবে: Browser Console ও Node.js

```
performance.mark('filter-start');
const items = Array.from({ length: 50_000 }, (_, i) => i);
const even = items.filter((n) => n % 2 === 0);
performance.mark('filter-end');

const entry = performance.measure('filter', 'filter-start', 'filter-end');
console.log(entry.name, entry.entryType, even.length);
console.log(performance.getEntriesByName('filter').length, typeof entry.duration);
```

Output

```
filter measure 25000
1 number
```

`duration`-এর মান ছাপা হয়নি, কারণ তা প্রতিবার ও প্রতি যন্ত্রে আলাদা; এই বইয়ের স্বয়ংক্রিয় যাচাই শুধু স্থির অংশ মেলায় (অধ্যায় ১৯-এর নীতি: সময় কখনো Pass/Fail শর্ত নয়)। একই Code Browser ও Node.js দুটিতেই চলে।

`PerformanceObserver` Entry তৈরি হওয়ার সঙ্গে সঙ্গে জানায়, তাই মাপার Code ও Report-এর Code আলাদা রাখা যায়: যে Module কাজ করে সে শুধু `mark` / `measure` বসায়; অন্য কোথাও একজন Observer সেগুলো জমা বা পাঠায় (যেমন Analytics-এ)।

observer.mjs · JavaScript — চলবে: Node.js

```
import { PerformanceObserver, performance } from 'node:perf_hooks';

const seen = [];
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach((entry) => seen.push(entry.name));
});
observer.observe({ entryTypes: ['measure'] });

for (const step of ['load', 'index', 'search']) {
  performance.mark(`${step}-start`);
  performance.measure(step, `${step}-start`);
}
```

```
setTimeout(() => {
  observer.disconnect();
  console.log(seen.join(' > '));
}, 0);
```

Output

```
load > index > search
```

Observer-এর Callback পরে, একসঙ্গে (Batch-এ) আসে—তাই ফল পড়া হয়েছে একটি `setTimeout`-এর ভেতরে। `measure`-এ শেষ চিহ্ন না দিলে "এখন" পর্যন্ত মাপা হয়।

২৩.৩ CPU Profile: সময় আসলে কোথায় যায়

User Timing বলে **কতক্ষণ**; Profiler বলে **কোথায়**। CPU Profiler প্রতি কয়েক মাইক্রোসেকেন্ডে দেখে নেয় এই মুহূর্তে কোন Function চলছে (Sampling)। যে Function-এ সবচেয়ে বেশি Sample, সেখানেই সবচেয়ে বেশি সময়। দুটি পরিমাপ গুরুত্বপূর্ণ:

পরিমাপ	অর্থ	কাজে লাগে
Self Time	Function নিজের Code-এ কাটানো সময়	সবচেয়ে ব্যয়বহুল লাইন খোঁজা
Total Time	নিজের ও তার ডাকা সব Function-এর সময়	কোন বড় কাজ দায়ী

Node.js-এ Profile নেওয়ার তিনটি উপায়: `node --cpu-prof script.mjs` (একটি `.cpuprofile` File লেখে, যা Chrome DevTools-এ খোলা যায়), `node --inspect` দিয়ে DevTools যুক্ত করা, অথবা Program-এর ভেতর থেকে `node:inspector`-এর Profiler। Project-এর `profile.mjs` তৃতীয়টি ব্যবহার করে, এবং `profile-report.mjs` Profile-এর JSON পড়ে Self Time-এর তালিকা বানায়। এই বইয়ের লেখার সময় একটি যন্ত্রে (Intel Core i5-6600, Windows 10, Node.js v22.16.0) এক চালানোর আসল Output:

Terminal

```
Node.js v22.16.0, win32; ১৫০টি করে খোঁজ
naive    median 8.293 ms
indexed  median 0.392 ms
সবচেয়ে বেশি Self Time (CPU Profile):
  68.6%  words (search.mjs:3)
  16.0%  (search.mjs)
  7.0%  naiveSearch (search.mjs:17)
  5.1%  (garbage collector) (native:0)
```

Profile একটি স্পষ্ট গল্প বলে: মোট সময়ের প্রায় দুই-তৃতীয়াংশ যায় `words` -এ—যে Function লেখাকে শব্দে ভাগ করে। কারণ সরল খোঁজ প্রতিটি প্রশ্নে ১২,০০০ Record-এর প্রতিটির লেখা **আবার** ভাগ করে। কোন Loop বা কোন তুলনা ধীর, তা অনুমান করলে আমরা হয়তো `startsWith` বা `filter` নিয়ে সময় নষ্ট করতাম। সমাধান তাই একটি: শব্দগুলো একবার ভাগ করে একটি **Index**-এ রাখা—Project-এর `buildIndex` —এবং Median ৮.৩ থেকে ০.৪ ms। সংখ্যাগুলো আপনার যন্ত্রে আলাদা হবে; অনুপাত ও Profile-এর গঠন দেখুন। "(garbage collector)" সারিটিও লক্ষ করুন: প্রতিবার নতুন Array তৈরি GC-কেও কাজ দেয়।

২৩.8 Event Loop Delay এবং Memory

একটি Server-এ "ধীর" মানে প্রায়ই Event Loop আটকে থাকা: একটি Request-এর ভারী হিসাব চলার সময় বাকি সব Request অপেক্ষা করে। `monitorEventLoopDelay` প্রতিটি Event Loop চক্রের দেরি একটি Histogram-এ জমায়:

loop-delay.mjs · JavaScript — চলবে: Node.js

```
import { monitorEventLoopDelay } from 'node:perf_hooks';

const histogram = monitorEventLoopDelay({ resolution: 10 });
histogram.enable();

function blockFor(ms) {
  const end = Date.now() + ms;
  while (Date.now() < end) {
    // ইচ্ছাকৃতভাবে Event Loop আটকে রাখা (শিক্ষামূলক প্রদর্শন)
  }
}

setTimeout(() => blockFor(200), 50);
setTimeout(() => {
  histogram.disable();
  const ms = (ns) => (ns / 1e6).toFixed(1);
  console.log(`median ${ms(histogram.percentile(50))} ms, `
    + `p99 ${ms(histogram.percentile(99))} ms, max ${ms(histogram.max)} ms`);
}, 500);
```

এর সংখ্যা প্রতিবার আলাদা, তাই এখানে কোনো যাচাই-করা Output Panel নেই; এই বইয়ের লেখার সময় এক চালানোর আসল ফল:

Terminal

```
median 15.9 ms, p99 200.1 ms, max 200.1 ms
```

Median প্রায় ১৬ ms—এই Windows যন্ত্রে Timer-এর স্বাভাবিক সূক্ষ্মতা (প্রায় ১৫.৬ ms)-র কাছাকাছি—কিন্তু p99 ও সর্বোচ্চ প্রায় ২০০ ms: সেই একটি আটকে থাকার সময়টুকু কোনো Request উত্তর পায়নি। Production-এ এই Histogram-এর p99 নিয়মিত Monitoring-এ পাঠানো হয়।

Browser-এ একই প্রশ্নের উত্তর দেয় **Long Task API**: Main Thread ৫০ ms-এর বেশি আটকে থাকলে একটি `longtask` Entry তৈরি হয়, যা `PerformanceObserver` দিয়ে ধরা যায়:

long-task.js · JavaScript — চলবে: Browser

```
const seen = [];
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach((entry) => seen.push(entry.entryType));
});
observer.observe({ type: 'longtask', buffered: true });

setTimeout(() => {
  const end = performance.now() + 120;
  while (performance.now() < end) {
    // ১২০ ms Main Thread আটকে রাখা (শিক্ষামূলক প্রদর্শন)
  }
}, 50);

setTimeout(() => {
  observer.disconnect();
  console.log(`Long Task ধরা পড়েছে: ${seen.includes('longtask')}`);
}, 500);
```

Output

```
Long Task ধরা পড়েছে: true
```

১২০ ms-এর আটকে থাকা ৫০ ms-এর সংজ্ঞার অনেক ওপরে, তাই ফল স্থির। বাস্তব Website-এ এই Observer প্রকৃত ব্যবহারকারীর Browser থেকে Long Task-এর সংখ্যা ও সময় সংগ্রহ করে (Real User Monitoring)—Lab-এর একটি দ্রুত যন্ত্রে যা কখনো ধরা পড়ে না, ধীর ফোনে তা নিয়মিত। Google-এর **INP** (Interaction to Next Paint) পরিমাপ ঠিক এই সমস্যারই ফল মাপে: ব্যবহারকারীর Click বা চাপ থেকে পরের ছবি আঁকা পর্যন্ত সময়, যা Long Task থাকলে বেড়ে যায়। নতুন Chrome-এ আরও বিস্তারিত `long-animation-frame` Entry আছে, যা বলে দেয় কোন Script দায়ী।

Memory: Leak প্রমাণের নির্ভরযোগ্য উপায় হলো Garbage Collection চালিয়ে তারপর Heap মাপা—নইলে "Memory বেশি" মানে হয়তো GC এখনো চলেনি। নিচের **শিক্ষামূলক প্রদর্শন** দুটি একই রকম Cache তুলনা করে: একটি সীমাহীন, একটি সীমিত (অধ্যায় ১৯-এর LRU-র নীতি)। GC হাতে চালাতে এখানে V8-এর একটি Flag Program-এর ভেতর থেকে চালু করা হয়েছে—শুধু পরীক্ষার জন্য, Production-এ নয়।

heap.mjs · JavaScript — চলবে: Node.js

```
import { setFlagsFromString } from 'node:v8';
import { runInNewContext } from 'node:vm';

setFlagsFromString('--expose-gc');
const gc = runInNewContext('gc');
const heapMB = () => {
  gc();
```

```
  return process.memoryUsage().heapUsed / 1024 / 1024;
};

function fillCache(cache, limit) {
  for (let i = 0; i < 400; i += 1) {
    cache.set(`report-${i}`, new Array(10_000).fill(i));
    if (cache.size > limit) {
      cache.delete(cache.keys().next().value);
    }
  }
}

const before = heapMB();
const unbounded = new Map();
fillCache(unbounded, Infinity);
const afterUnbounded = heapMB();
const bounded = new Map();
fillCache(bounded, 20);
const afterBounded = heapMB();

const grewBy = (a, b) => b - a;
console.log(`সীমাহীন Cache-এ Heap ৩০ MB-র বেশি বেড়েছে: ${grewBy(before, afterUnbounded) >
    30}`);
console.log(`সীমিত Cache-এ বাড়তি ৫ MB-র কম: ${grewBy(afterUnbounded, afterBounded) <
    5}`);
console.log(unbounded.size, bounded.size);
```

Output

```
সীমাহীন Cache-এ Heap ৩০ MB-র বেশি বেড়েছে: true
সীমিত Cache-এ বাড়তি ৫ MB-র কম: true
400 20
```

এই তুলনা সময় নয়, Memory: GC-র পরে যা থেকে যায় তা পৌঁছানো যায় এমন Object, তাই ফল স্থির। ৪০০টি Report, প্রতিটি দশ হাজার সংখ্যার Array (প্রায় ৮০ KB), সীমাহীন Cache-এ সবই জীবিত; সীমিত Cache-এ মাত্র ২০টি। Browser-এ একই কাজ DevTools-এর Memory Panel-এ দুটি Heap Snapshot তুলনা করে করা হয় ("Comparison" View-তে কোন ধরনের Object বেড়েছে)।

২৩.৫ Browser Rendering Pipeline

Browser-এ একটি পরিবর্তন পর্দায় পৌঁছাতে পাঁচটি ধাপ পেরোতে পারে:

ধাপ	কাজ	কী ট্রিগার করে
JavaScript	আপনার Code	Event, Timer

ধাপ	কাজ	কী ট্রিগার করে
Style	কোন Element-এ কোন CSS	Class, Style বদল
Layout	প্রতিটি Element-এর আকার ও অবস্থান	`width`, `height`, `top`, Font, লেখা
Paint	Pixel আঁকা	রং, ছায়া, লেখা
Composite	স্তরগুলো জোড়া (প্রায়ই GPU-তে)	`transform`, `opacity`

যে পরিবর্তন যত আগের ধাপ ট্রিগার করে, তত দামি: `height` বদলালে Layout, Paint, Composite সবই; `transform` বদলালে সাধারণত শুধু Composite। তাই Animation ও সরানোর কাজে `transform` ও `opacity`। পার্থক্যটি মাপা যায় Layout-এর ফল দিয়ে:

pipeline.js · JavaScript — চলবে: Browser

```
const box = document.createElement('div');
box.style.height = '50px';
const next = document.createElement('p');
next.textContent = 'পরের অংশ';
document.body.append(box, next);

const before = next.offsetTop;
box.style.transform = 'translateY(100px)';
const afterTransform = next.offsetTop;
box.style.height = '150px';
const afterHeight = next.offsetTop;

console.log(`transform-এর পর পরের অংশ সরেছে? ${afterTransform !== before}`);
console.log(`height-এর পর সরেছে? ${afterHeight !== before}, ${afterHeight -
    before}px`);
```

Output

```
transform-এর পর পরের অংশ সরেছে? false
height-এর পর সরেছে? true, 100px
```

`transform` বাক্সটিকে দৃশ্যত সরিয়েছে, কিন্তু Layout বদলায়নি—পরের অংশ যেখানে ছিল সেখানেই। `height` বদলানো পুরো Page-এর Layout নতুন করে হিসাব করিয়েছে। Project-এর Virtual List ঠিক এই কারণে সারিগুলো `transform: translateY(...)` দিয়ে সরায়। আরেকটি কার্যকর CSS: `content-visibility: auto`, যা পর্দার বাইরের অংশের Layout ও Paint পিছিয়ে দেয়।

২৩.৬ Virtual Scrolling

১২,০০০ সারি DOM-এ রাখলে প্রতিটি সারির জন্য Element, Style, Layout ও Memory—পর্দায় দেখা যায় মাত্র ১০টি। **Virtual Scrolling** শুধু দৃশ্যমান সারি (এবং দুই পাশে কয়েকটি বাড়তি, **Overscan**) আঁকে। কৌশলটির তিনটি অংশ:

১. **Spacer:** একটি খালি Element, যার উচ্চতা সব সারির মোট উচ্চতা (১২,০০০ × ৪৮px)। এতে Scrollbar-এর আকার ও অবস্থান সঠিক থাকে।

২. **হিসাব:** `scrollTop` থেকে প্রথম ও শেষ দৃশ্যমান সারির Index—একটি বিশুদ্ধ Function, তাই Browser ছাড়াই Test করা যায়।

৩. **অবস্থান:** আঁকা সারিগুলো `translateY(start × rowHeight)` দিয়ে সঠিক জায়গায়।

virtual-demo.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
import { visibleRange } from '../project/travel-search/src/virtual.mjs';

const list = { viewport: 480, rowHeight: 48, total: 12_000 };
for (const scrollTop of [0, 24, 48_000, 575_520]) {
  const { start, end } = visibleRange({ ...list, scrollTop });
  console.log(`scrollTop ${scrollTop}: সারি ${start} থেকে ${end - 1}, মোট ${end -
      start}`);
}
```

Output

```
scrollTop 0: সারি 0 থেকে 14, মোট 15
scrollTop 24: সারি 0 থেকে 15, মোট 16
scrollTop 48000: সারি 995 থেকে 1014, মোট 20
scrollTop 575520: সারি 11985 থেকে 11999, মোট 15
```

তালিকা ১২,০০০ বা দশ লাখ—DOM-এ সর্বোচ্চ ২০টি সারি। সীমাবদ্ধতা: এই সরল রূপ সব সারির উচ্চতা সমান ধরে; ভিন্ন উচ্চতার সারি, Keyboard দিয়ে চলাচল, Screen Reader-এর জন্য মোট সংখ্যা জানানো (`aria-rowcount`), এবং Browser-এর "Find in page" (যা না-আঁকা সারি খুঁজে পায় না)—এগুলো বাস্তব Data Grid Library সামলায়। তাই প্রশ্নটি সব সময় করুন: সত্যিই কি হাজার সারি দেখাতে হবে, নাকি Pagination বা ভালো খোঁজ যথেষ্ট?

২৩.৭ Web Worker: Main Thread মুক্ত রাখা

Browser-এ JavaScript, Layout, Paint এবং ব্যবহারকারীর Click—সবই সাধারণত একই **Main Thread**-এ। ভারী হিসাব সেখানে চললে Page জমে যায় (অধ্যায় ১৯-এ কাজ টুকরোয় ভাগ করে সমাধান দেখেছিলেন)। **Web Worker** একটি আলাদা Thread, যার নিজস্ব Global Scope: DOM, `window` বা `document` নেই, আছে `postMessage` ও `message` Event। Module Worker (`{ type: 'module' }`) সাধারণ `import` ব্যবহার করতে পারে।

main.mjs · JavaScript — চলবে: Browser

```
const url = new URL('./worker.mjs', import.meta.url);
const worker = new Worker(url, { type: 'module' });

worker.addEventListener('message', ({ data }) => {
  console.log(`Worker-এর যোগফল: ${data}`);
  worker.terminate();
});

worker.postMessage(10_000_000);
console.log('postMessage সঙ্গে সঙ্গে ফিরেছে; Main Thread মুক্ত');
```

worker.mjs · JavaScript — চলবে: Browser

```
addEventListener('message', ({ data: limit }) => {
  let sum = 0;
  for (let n = 1; n <= limit; n += 1) {
    sum += n;
  }
  postMessage(sum);
});
```

Chrome-এ চালালে:

Output

```
postMessage সঙ্গে সঙ্গে ফিরেছে; Main Thread মুক্ত
Worker-এর যোগফল: 50000005000000
```

ক্রমটিই মূল কথা: `postMessage` হিসাবের জন্য অপেক্ষা করে না, তাই Main Thread পরের লাইনে চলে যায় এবং হিসাবের সময় Click ও Scroll-এ সাড়া দিতে পারে। ফল আসে পরে, একটি Message হিসেবে।

Message-এর খরচ: `postMessage`-এর তথ্য **Structured Clone** হয়—পুরো একটি Copy। বড় তথ্যে এই Copy-ই ধীর হতে পারে। `ArrayBuffer`-এর মতো কিছু Object **Transfer** করা যায়: Copy নয়, মালিকানা হস্তান্তর, এবং পাঠানোর পর মূল Object ব্যবহার-অযোগ্য:

transfer.js · JavaScript — চলবে: Browser Console ও Node.js

```
const buffer = new ArrayBuffer(8 * 1024 * 1024);

const copy = structuredClone(buffer);
console.log(`Clone-এর পরে: মূল ${buffer.byteLength}, Copy ${copy.byteLength}`);

const moved = structuredClone(buffer, { transfer: [buffer] });
console.log(`Transfer-এর পরে: মূল ${buffer.byteLength}, নতুন ${moved.byteLength}`);
```

Output

```
Clone-এর পরে: মূল 8388608, Copy 8388608
Transfer-এর পরে: মূল 0, নতুন 8388608
```

`worker.postMessage(data, [buffer])` একই নিয়মে কাজ করে। Project-এর Worker একটি ভিন্ন কৌশল ব্যবহার করে: তথ্য Main Thread-এ পাঠানোই হয় না—Worker নিজেই তৈরি ও Index করে, এবং শুধু ফল ফেরত পাঠায়। সীমাবদ্ধতা: প্রতিটি খোঁজে ফলের তালিকা (সর্বোচ্চ ১২,০০০টি ছোট Object) Clone হয়; আরও বড় তথ্যে শুধু ID বা শুধু দৃশ্যমান পাতা পাঠানো ভালো।

২৩.৮ Project: TravelSearch Explorer

TravelSearch Explorer একটি ভ্রমণ সংস্থার প্যাকেজ খোঁজার Page: ১২,০০০টি কাল্পনিক প্যাকেজ (Seed থেকে, তাই প্রতিবার একই), শব্দের শুরু দিয়ে খোঁজ ("syl tea"), দাম বা Rating-এ সাজানো, এবং একটি Virtual List। খোঁজ চলে একটি Web Worker-এ। অধ্যায় ১৯-এর PerfLab ছিল Node.js-এর Benchmark Harness; এটি একটি পূর্ণ Browser Application, যেখানে Profiling থেকে পাওয়া সিদ্ধান্ত (Index) প্রয়োগ করা হয়েছে এবং আচরণ আসল Browser-এ যাচাই করা হয়েছে।

> **নোট:** শিক্ষামূলক Demo। তথ্য কাল্পনিক; Virtual List সমান-উচ্চতার সারি ধরে নেয় এবং পূর্ণ Accessibility (Keyboard চলাচল, `aria-rowcount`) দেয় না।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-search/
├── index.html, style.css
├── serve.mjs               স্থানীয় Static Server (Worker-এর জন্য HTTP)
├── profile.mjs             Profiling Lab (User Timing ও CPU Profile)
├── src/
│   ├── data.mjs            Seed-যুক্ত ১২,০০০ Record
│   ├── search.mjs          সরল ও Index-ভিত্তিক খোঁজ
│   ├── virtual.mjs         দৃশ্যমান সারির হিসাব
│   ├── search-worker.mjs   Web Worker
│   ├── app.mjs             UI: Debounce, Virtual List, Worker
│   └── profile-report.mjs  CPU Profile থেকে Self Time
└── tests/                  search, virtual, e2e (+ support/browser.mjs)
```

src/data.mjs ও src/search.mjs

data.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// ১২,০০০টি কাল্পনিক ভ্রমণ-প্যাকেজ, Seed থেকে: প্রতিবার হুবহু একই তথ্য।
```

```
const PLACES = ['Dhaka', 'Sylhet', 'Bandarban', 'Rangamati', 'Khulna',
  'Kuakata', 'Srimangal', 'Rajshahi', 'Bogura', 'Saint Martin',
  'Kathmandu', 'Darjeeling', 'Bangkok', 'Kuala Lumpur', 'Dubai'];
const KINDS = ['Hotel', 'Resort', 'Tour', 'Cruise', 'Homestay'];
const WORDS = ['River', 'Hill', 'Lake', 'Garden', 'Sea', 'Tea', 'Royal',
  'Green', 'Golden', 'Silent'];

export function createRandom(seed) {
  let state = seed >>> 0;
  return () => {
    state = (state + 0x6d2b79f5) >>> 0;
    let t = Math.imul(state ^ (state >>> 15), state | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

export function generateRecords(count = 12_000, seed = 23) {
  const random = createRandom(seed);
  const pick = (list) => list[Math.floor(random() * list.length)];
  return Array.from({ length: count }, (_, i) => {
    const place = pick(PLACES);
    const kind = pick(KINDS);
    return {
      id: i + 1,
      title: `${pick(WORDS)} ${pick(WORDS)} ${kind}`,
      place,
      kind,
      price: 1_500 + Math.floor(random() * 60) * 500,
      rating: Math.round((3 + random() * 2) * 10) / 10,
    };
  });
}
```

search.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Browser Console ও Node.js

```
// দুটি খোঁজ: সরল (প্রতিবার সব Record স্ক্যান) এবং Index-ভিত্তিক।
// দুটির ফল হুবহু এক হতে হবে; Test তা যাচাই করে।
const words = (text) => text.toLowerCase().split(/[^a-z0-9]+/).filter(Boolean);
const textOf = (r) => `${r.title} ${r.place} ${r.kind}`;

const SORTS = {
  relevance: (a, b) => a.id - b.id,
  price: (a, b) => a.price - b.price || a.id - b.id,
  rating: (a, b) => b.rating - a.rating || a.id - b.id,
};

function finish(records, { maxPrice = Infinity, sort = 'relevance' }) {
  return records.filter((r) => r.price <= maxPrice).sort(SORTS[sort]);
```

```
}

// প্রতিটি প্রশ্নের শব্দ কোনো Record-এর কোনো শব্দের শুরু হতে হবে।
export function naiveSearch(records, query, options = {}) {
  const terms = words(query);
  const hits = records.filter((r) => {
    const own = words(textOf(r));
    return terms.every((t) => own.some((w) => w.startsWith(t)));
  });
  return finish(hits, options);
}

// Index: সাজানো শব্দের তালিকা এবং প্রতিটি শব্দের Record-গুলো।
export function buildIndex(records) {
  const postings = new Map();
  for (const r of records) {
    for (const w of new Set(words(textOf(r)))) {
      if (!postings.has(w)) {
        postings.set(w, []);
      }
      postings.get(w).push(r);
    }
  }
  return { records, postings, sorted: [...postings.keys()].sort() };
}

// ...
```

search.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

// একটি Prefix-এর সব শব্দ: সাজানো তালিকায় Binary Search, তারপর পরপর।
function wordsWithPrefix(sorted, prefix) {
  let low = 0;
  let high = sorted.length;
  while (low < high) {
    const middle = (low + high) >>> 1;
    if (sorted[middle] < prefix) {
      low = middle + 1;
    } else {
      high = middle;
    }
  }
  const found = [];
  for (let i = low; i < sorted.length && sorted[i].startsWith(prefix); i += 1) {
    found.push(sorted[i]);
  }
  return found;
```

```
}

export function indexedSearch(index, query, options = {}) {
  let result = null;
  for (const term of words(query)) {
    const matches = new Set();
    for (const word of wordsWithPrefix(index.sorted, term)) {
      index.postings.get(word).forEach((r) => matches.add(r));
    }
    result = result === null
      ? matches
      : new Set([...result].filter((r) => matches.has(r)));
  }
  return finish(result === null ? index.records : [...result], options);
}
```

Index-এ প্রতিটি আলাদা শব্দের জন্য সেই শব্দযুক্ত Record-গুলোর তালিকা (**Inverted Index**, সার্চ ইঞ্জিনের মূল কাঠামো), আর শব্দগুলোর একটি সাজানো তালিকা। "syl" খুঁজতে সাজানো তালিকায় Binary Search দিয়ে প্রথম মিলের অবস্থান, তারপর পরপর যতক্ষণ শব্দ "syl" দিয়ে শুরু হয়। একাধিক শব্দে (`tea resort`) ফলগুলোর ছেদ (Intersection)। Index Record-গুলোর **Reference** রাখে, Copy নয়, তাই Memory দ্বিগুণ হয় না (একটি Test এটি যাচাই করে)। সরল খোঁজ (`naiveSearch`) রেখে দেওয়া হয়েছে উদ্দেশ্যমূলকভাবে: সেটি "সঠিক উত্তরের" মাপকাঠি।

src/virtual.mjs ও src/search-worker.mjs

virtual.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
// Virtual Scrolling-এর হিসাব: কোন সারিগুলো আঁকতে হবে। বিশুদ্ধ Function, তাই
// Browser ছাড়াই Test করা যায়।
export function visibleRange(options) {
  const { scrollTop, viewport, rowHeight, total, overscan = 5 } = options;
  if (total === 0) {
    return { start: 0, end: 0 };
  }
  const first = Math.floor(Math.max(0, scrollTop) / rowHeight);
  const last = Math.ceil((Math.max(0, scrollTop) + viewport) / rowHeight);
  return {
    start: Math.max(0, first - overscan),
    end: Math.min(total, last + overscan),
  };
}

export const offsetOf = (index, rowHeight) => index * rowHeight;
```

search-worker.mjs · JavaScript — চলবে: Browser

```
// Web Worker: আলাদা Thread-এ তথ্য তৈরি, Index ও খোঁজ। DOM-এ প্রবেশ নেই;
```

```
// Main Thread-এর সঙ্গে শুধু Message।
import { generateRecords } from './data.mjs';
import { buildIndex, indexedSearch } from './search.mjs';

performance.mark('index-start');
const index = buildIndex(generateRecords());
performance.mark('index-end');
const built = performance.measure('index', 'index-start', 'index-end');
postMessage({ type: 'ready', total: index.records.length, ms: built.duration });

addEventListener('message', ({ data }) => {
  const started = performance.now();
  const results = indexedSearch(index, data.query, data.options);
  postMessage({
    type: 'results',
    requestId: data.requestId,
    rows: results,
    ms: performance.now() - started,
  });
});
```

Worker চালু হয়েই তথ্য তৈরি করে Index বানায় এবং একটি `ready` Message পাঠায়; তারপর প্রতিটি প্রশ্নে খুঁজে ফল ও সময় (`ms`) পাঠায়। User Timing Worker-এর ভেতরেও কাজ করে।

src/app.mjs

app.mjs · JavaScript · অংশ ১ / ২ (Part 1 of 2) — চলবে: Browser

```
import { visibleRange, offsetOf } from './virtual.mjs';

const ROW_HEIGHT = 48;
const $ = (selector) => document.querySelector(selector);
const worker = new Worker(new URL('./search-worker.mjs', import.meta.url),
  { type: 'module' });
let rows = [];
let latestRequest = 0;
let frame = 0;
let timer;

// শুধু দৃশ্যমান সারি (এবং দুই পাশে কয়েকটি বাড়তি) DOM-এ।
function render() {
  frame = 0;
  const list = $('#list');
  const { start, end } = visibleRange({
    scrollTop: list.scrollTop, viewport: list.clientHeight,
    rowHeight: ROW_HEIGHT, total: rows.length,
  });
  $('#spacer').style.height = `${rows.length * ROW_HEIGHT}px`;
  $('#rows').style.transform = `translateY(${offsetOf(start, ROW_HEIGHT)}px)`;
```

```
  $('#rows').replaceChildren(...rows.slice(start, end).map((r) => {
    const item = document.createElement('li');
    item.textContent = `${r.title} · ${r.place} · ৳${r.price} · ${r.rating}`;
    return item;
  }));
}

function search() {
  latestRequest += 1;
  worker.postMessage({
    requestId: latestRequest,
    query: $('#query').value,
    options: { sort: $('#sort').value },
  });
}

// ...
```

app.mjs · JavaScript · অংশ ২ / ২ (Part 2 of 2) · চলবে: Browser

```
// ...

worker.addEventListener('message', ({ data }) => {
  if (data.type === 'ready') {
    $('#status').textContent = `${data.total}টি প্যাকেজ প্রস্তুত`;
    search();
    return;
  }
  // ধীর পুরোনো উত্তর নতুন উত্তরকে মুছে দেবে না।
  if (data.requestId !== latestRequest) {
    return;
  }
  rows = data.rows;
  $('#list').scrollTop = 0;
  render();
  $('#count').textContent = `${rows.length}টি ফল · খোঁজ ${data.ms.toFixed(1)} ms`;
  $('#count').dataset.request = String(data.requestId);
});

$('#query').addEventListener('input', () => {
  clearTimeout(timer);
  timer = setTimeout(search, 150);
});
$('#sort').addEventListener('change', search);
// একটি Frame-এ একবারই আঁকা, যত Scroll Event-ই আসুক।
$('#list').addEventListener('scroll', () => {
  frame ||= requestAnimationFrame(render);
}, { passive: true });
```

চারটি Performance-এর সিদ্ধান্ত এখানে: খোঁজ Worker-এ; লেখার সময় ১৫০ ms Debounce (অধ্যায় ১৪); Scroll Event-এ সরাসরি না এঁকে `requestAnimationFrame`-এ, এবং একটি Frame-এ একবারই (`frame ||=`); আর প্রতিটি অনুরোধের একটি ক্রমিক নম্বর (`requestId`)—পুরোনো ধীর উত্তর পরে এলে তা উপেক্ষিত, যাতে "sy"-এর উত্তর "sylhet"-এর উত্তরকে মুছে না দেয়। এই শেষ সমস্যাটির নাম **Race Condition**, এবং Asynchronous খোঁজে এটি প্রায় সব সময় থাকে।

index.html ও style.css

index.html · HTML — চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>TravelSearch Explorer</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="src/app.mjs"></script>
</head>
<body>
  <h1>TravelSearch Explorer <small>শিক্ষামূলক Demo</small></h1>
  <p id="status" role="status">তথ্য তৈরি হচ্ছে...</p>
  <label>খুঁজুন <input id="query" placeholder="যেমন: sylhet tea"></label>
  <label>সাজান <select id="sort">
    <option value="relevance">ক্রমিক</option>
    <option value="price">দাম</option>
    <option value="rating">Rating</option>
  </select></label>
  <p id="count"></p>
  <div id="list">
    <div id="spacer"></div>
    <ul id="rows"></ul>
  </div>
</body>
</html>
```

style.css · CSS — চলবে: Browser

```
body {
  font-family: system-ui, "Nirmala UI", sans-serif;
  max-width: 48rem;
  margin: 2rem auto;
  padding: 0 1rem;
}

/* Scroll-এর পাত্র: নির্দিষ্ট উচ্চতা, ভেতরে একটি লম্বা Spacer। */
#list {
  position: relative;
```

```
  height: 480px;
  overflow-y: auto;
  border: 1px solid #cbd5e1;
}

/* সারিগুলো Spacer-এর ওপরে; transform দিয়ে সরানো, Layout ছাড়াই। */
#rows {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  margin: 0;
  padding: 0;
  list-style: none;
  will-change: transform;
}

#rows li {
  height: 48px;
  line-height: 48px;
  padding: 0 0.75rem;
  border-bottom: 1px solid #e2e8f0;
  overflow: hidden;
  white-space: nowrap;
}
```

Profiling Lab: profile.mjs

profile.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// Profiling Lab (Node.js): User Timing দিয়ে প্রতিটি খোঁজের সময় এবং
// Inspector-এর CPU Profiler দিয়ে কোন Function সবচেয়ে বেশি সময় নেয়।
// সংখ্যাগুলো যন্ত্রভেদে বদলায়; এই Script কোনো Test-এর অংশ নয়।
import { PerformanceObserver, performance } from 'node:perf_hooks';
import { Session } from 'node:inspector/promises';
import { generateRecords } from './src/data.mjs';
import { buildIndex, naiveSearch, indexedSearch } from './src/search.mjs';
import { topFunctions } from './src/profile-report.mjs';

const durations = { naive: [], indexed: [] };
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach((e) => durations[e.name].push(e.duration));
});
observer.observe({ entryTypes: ['measure'] });

function timed(name, fn) {
  performance.mark(`${name}-start`);
  fn();
  performance.measure(name, `${name}-start`);
```

```
}

// ...
```

profile.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

const records = generateRecords();
const index = buildIndex(records);
const queries = ['sylhet', 'tea resort', 'go', 'river hill', 'dubai cruise'];

const session = new Session();
session.connect();
await session.post('Profiler.enable');
await session.post('Profiler.start');
for (let round = 0; round < 30; round += 1) {
  for (const q of queries) {
    timed('naive', () => naiveSearch(records, q));
    timed('indexed', () => indexedSearch(index, q));
  }
}
const { profile } = await session.post('Profiler.stop');
await new Promise((resolve) => setImmediate(resolve));
observer.disconnect();

const median = (list) => list.toSorted((a, b) => a - b)[list.length >> 1];
console.log(`Node.js ${process.version}, ${process.platform}; ১৫০টি করে খোঁজ`);
for (const [name, list] of Object.entries(durations)) {
  console.log(`${name.padEnd(8)} median ${median(list).toFixed(3)} ms`);
}
console.log('সবচেয়ে বেশি Self Time (CPU Profile):');
for (const f of topFunctions(profile, 4)) {
  console.log(`  ${f.percent}%  ${f.name}`);
}
```

profile-report.mjs · JavaScript চলবে: Node.js

```
// V8-এর CPU Profile (.cpuprofile-এর একই JSON) থেকে Self Time-এর তালিকা।
// প্রতিটি Sample মানে: সেই মুহূর্তে কোন Function চলছিল।
export function topFunctions(profile, limit = 5) {
  const counts = new Map();
  const byId = new Map(profile.nodes.map((node) => [node.id, node]));
  for (const id of profile.samples) {
    const { functionName, url, lineNumber } = byId.get(id).callFrame;
    const file = url.split('/').pop();
    const name = functionName
      ? `${functionName} (${file || 'native'}:${lineNumber + 1})`
      : `(${file || 'idle বা অন্যান্য'})`;
```

```
    counts.set(name, (counts.get(name) ?? 0) + 1);
  }
  const total = profile.samples.length;
  return [...counts]
    .sort((a, b) => b[1] - a[1])
    .slice(0, limit)
    .map(([name, n]) => ({ name, percent: ((n / total) * 100).toFixed(1) }));
}
```

এর আসল Output অনুচ্ছেদ ২৩.৩-এ। `profile.mjs` ইচ্ছা করে Automated Test-এর অংশ নয়: সময় যন্ত্রভেদে বদলায়।

Test: সঠিকতা আগে, তারপর গতি

search.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// Optimization-এর প্রথম শর্ত: দ্রুত সংস্করণের ফল ধীর সংস্করণের হুবহু সমান।
// কোনো Test সময় মাপে না।
import test from 'node:test';
import assert from 'node:assert/strict';
import { generateRecords, createRandom } from '../src/data.mjs';
import { buildIndex, naiveSearch, indexedSearch } from '../src/search.mjs';

const records = generateRecords();
const index = buildIndex(records);
const ids = (list) => list.map((r) => r.id);

// ...

test('Property: ৩০০টি এলোমেলো প্রশ্নে দুই খোঁজ সমান', () => {
  const random = createRandom(99);
  const vocabulary = ['sy', 'tea', 'hill', 'resort', 'dha', 'g', 'cruise',
    'bang', 'lake', 'home', 'k', 'river', 'royal'];
  for (let i = 0; i < 300; i += 1) {
    const size = 1 + Math.floor(random() * 3);
    const q = Array.from({ length: size },
      () => vocabulary[Math.floor(random() * vocabulary.length)]).join(' ');
    const fast = ids(indexedSearch(index, q));
    assert.deepEqual(fast, ids(naiveSearch(records, q)), q);
  }
});

// ...
```

e2e.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

  await t.test('শেষ পর্যন্ত Scroll: শেষ Record আঁকা হয়', async () => {
    await page.eval(`(() => { const list = document.querySelector('#list');
      list.scrollTop = list.scrollHeight; })()`);
    await page.eval(
      'new Promise((r) => requestAnimationFrame(() => setTimeout(r, 50)))');
    const last = await page.eval(
      "document.querySelector('#rows li:last-child').textContent");
    assert.ok(last.startsWith(records.at(-1).title), last);
    assert.ok(await domRows() <= 20);
  });

// ...
```

Index একটি Optimization, এবং প্রতিটি Optimization-এর প্রথম শর্ত: ফল বদলাবে না। তাই Test-এর কেন্দ্র একটি Property (অধ্যায় ২২): ৩০০টি Seed-যুক্ত এলোমেলো প্রশ্নে দুই খোঁজ হুবহু সমান। E2E Test আসল Browser-এ তিনটি আচরণ যাচাই করে, কোনো সময় না মেপে: ১২,০০০ ফলেও DOM-এ সর্বোচ্চ ২০টি সারি, একদম নিচে Scroll করলে শেষ Record আঁকা হয়, এবং Worker-এর খোঁজ ও সাজানো Node.js-এর সরল খোঁজের সঙ্গে মেলে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-search
node --test
node profile.mjs
node serve.mjs
```

`node --test` ১৫টি Test চালায় (`search` ৫, `virtual` ৬, এবং `e2e` ৪—একটি Parent ও তিনটি Subtest); শেষে `# pass 15`, `# fail 0`। Chrome বা Edge না থাকলে E2E কারণসহ Skip হয়। `node serve.mjs` একটি স্থানীয় ঠিকানা দেখায় (Module Worker `file://` থেকে চলে না); DevTools-এর Performance Panel-এ Record করে লিখুন ও Scroll করুন, এবং Elements Panel-এ দেখুন `#rows`-এ কতগুলো `li`।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
"তথ্য তৈরি হচ্ছে…" থেকে নড়ে না	Page `file://` থেকে খোলা; Worker লোড হয়নি	`node serve.mjs`
Scroll-এ ফাঁকা জায়গা দেখা যায়	Overscan খুব কম বা সারির উচ্চতা CSS ও JS-এ আলাদা	`ROW_HEIGHT` ও CSS মেলান

লক্ষণ	সম্ভাব্য কারণ	করণীয়
লেখার সময় পুরোনো ফল ফিরে আসে	`requestId` যাচাই নেই	পুরোনো উত্তর বাদ
Index-এর ফল সরল খোঁজ থেকে আলাদা	Tokenizing দুই জায়গায় ভিন্ন	একই `words` Function
Profile-এ "(idle)" বেশি	Program অপেক্ষা করছে, হিসাব নয়	I/O বা Timer দেখুন

২৩.৯ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
Profile ছাড়া অনুমানে Optimize	ভুল জায়গায় সময়	আগে CPU Profile
Total Time দেখে Self Time ভুলে যাওয়া	দায়ী Function অস্পষ্ট	দুটিই দেখুন
GC না চালিয়ে Memory মাপা	বিভ্রান্তিকর সংখ্যা	GC-র পরে মাপ, Snapshot তুলনা
Animation-এ `top` / `height` বদলানো	প্রতি Frame-এ Layout	`transform`, `opacity`
হাজারো সারি DOM-এ	ধীর Rendering, বেশি Memory	Virtual Scrolling বা Pagination
Virtual List-এ উচ্চতা অনুমান ভুল	ফাঁকা জায়গা, লাফ	CSS ও JS-এ একই উচ্চতা
Worker-এ বিশাল তথ্য বারবার পাঠানো	Clone-এর খরচ	Worker-এ তথ্য রাখা, Transfer
Async খোঁজে অনুরোধের ক্রম না দেখা	পুরোনো ফল নতুনকে মুছে দেয়	`requestId`
Optimization-এর পর ফল যাচাই না করা	দ্রুত কিন্তু ভুল	সরল সংস্করণের সঙ্গে Property Test

২৩.১০ Debugging নির্দেশিকা

১. **লক্ষণ মাপুন:** কোন কাজ, কত সময়, কোন যন্ত্রে—User Timing দিয়ে নাম দিন।
২. **CPU Profile নিন:** `node --cpu-prof` বা DevTools-এর Performance Panel; Self Time অনুযায়ী সাজান।
৩. **Main Thread-এর Long Task:** Performance Panel-এ লাল কোণযুক্ত কাজ; সেটি কি Worker-এ যেতে পারে?
৪. **Rendering:** Panel-এ বেগুনি (Layout) ও সবুজ (Paint) ঘটনা; "Rendering" Tab-এ "Paint flashing" চালু করে দেখুন কী নতুন করে আঁকা হচ্ছে।
৫. **Memory:** দুটি Heap Snapshot, মাঝে কাজটি কয়েকবার; Comparison View-তে বাড়তে থাকা Object।
৬. **Server:** Event Loop Delay-এর p99; বেড়ে গেলে কোন Request চলছিল দেখুন।

২৩.১১ পেশাদার অভ্যাস

- **Profile আগে, Code পরে;** এবং পরিবর্তনের পর আবার Profile।
- **নিজের Code-এ নামযুক্ত মাপের বিন্দু** (User Timing), যাতে Profile-এ খুঁজে পাওয়া যায়।
- **প্রতিটি Optimization-এর সঙ্গে সঠিকতার Test,** সরল সংস্করণকে মাপকাঠি রেখে।
- **Main Thread-এ শুধু UI;** ভারী হিসাব Worker-এ।
- **Animation-এ শুধু `transform` ও `opacity`।**
- **বড় তালিকা আঁকার আগে প্রশ্ন করুন:** Pagination বা ভালো Filter কি যথেষ্ট?
- **ফলের সঙ্গে পরিবেশ লিখুন;** একটি সংখ্যা নয়, অনুপাত ও প্রবণতা।

২৩.১২ অনুশীলনী

প্রাথমিক স্তর

১. `performance.mark` ও `measure` দিয়ে একটি Array সাজানোর কাজের নামযুক্ত পরিমাপ তৈরি করুন, এবং Entry-র নাম ও ধরন ছাপুন।

২. Self Time ও Total Time-এর পার্থক্য একটি উদাহরণ দিয়ে ব্যাখ্যা করুন।

৩. নিচের কোনগুলো সাধারণত শুধু Composite ধাপ ট্রিগার করে: `width`, `transform`, `opacity`, `color`, `top`?

৪. `visibleRange` দিয়ে হিসাব করুন: ৬০০px উঁচু তালিকা, ৪০px সারি, ৫০০টি সারি, `scrollTop` ১০০০, Overscan ৩।

৫. Web Worker কেন `document` ব্যবহার করতে পারে না? এক বাক্যে।

মধ্যম স্তর

৬. `PerformanceObserver` দিয়ে শুধু ২০ ms-এর বেশি দীর্ঘ `measure` Entry-র নাম সংগ্রহ করুন (Node.js), নির্দিষ্টভাবে দীর্ঘ একটি কাজ দিয়ে।

৭. Project-এর `topFunctions` দিয়ে একটি ছোট Program-এর CPU Profile বিশ্লেষণ করুন, যেখানে একটি Function অন্যটির চেয়ে অনেক বেশি কাজ করে।

৮. একটি Search Worker-এ "অনুরোধ বাতিল" যোগের নকশা করুন: নতুন প্রশ্ন এলে পুরোনোর ফল পাঠানোই হবে না।

৯. Event Listener-এর একটি Leak তৈরি ও সমাধান করুন: প্রতিবার নতুন Object যুক্ত করা Listener, এবং `AbortController` দিয়ে পরিষ্কার—Heap মেপে দেখান।

চ্যালেঞ্জ স্তর

১০. ভিন্ন উচ্চতার সারির জন্য `visibleRange`-এর একটি সংস্করণ লিখুন: প্রতিটি সারির উচ্চতার Array থেকে Prefix Sum, তারপর Binary Search।

১১. Project-এর Index-এ ফলাফলকে Rating দিয়ে সাজানোর খরচ কমান: প্রতিবার সাজানোর বদলে Index তৈরির সময় একবার সাজিয়ে রাখার নকশা, এবং ফল যে বদলায় না তার Test।

১২. Worker থেকে ফলের পুরো Object-এর বদলে শুধু ID পাঠানোর একটি সংস্করণের নকশা করুন: Main Thread কীভাবে Record পাবে, এবং কী লাভ-ক্ষতি।

২৩.১৩ অনুশীলনীর সমাধান

সমাধান ১

sort-timing.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = Array.from({ length: 20_000 }, (_, i) => (i * 7919) % 20_000);

performance.mark('sort-start');
const sorted = numbers.toSorted((a, b) => a - b);
performance.mark('sort-end');

const [entry] = performance.getEntriesByName(
  performance.measure('sort', 'sort-start', 'sort-end').name);
console.log(entry.name, entry.entryType, sorted[0], sorted.at(-1));
```

Output

```
sort measure 0 19999
```

সমাধান ২ — ধরুন `buildReport` নিজে একটি Loop চালায় (১০ ms) এবং `formatRow`-কে ডাকে, যা মোট ৯০ ms নেয়। `buildReport`-এর Self Time ১০ ms, Total Time ১০০ ms; `formatRow`-এর Self ও Total দুটিই ৯০ ms (যদি সে অন্য কিছু না ডাকে)। Optimize করতে হবে `formatRow`-কে—Total Time একা দেখলে মনে হতো `buildReport` দায়ী।

সমাধান ৩ — `transform` ও `opacity` (Browser যখন সেগুলো আলাদা Composite স্তরে রাখে)। `width` ও `top` Layout ট্রিগার করে; `color` Paint।

সমাধান ৪

range.mjs · JavaScript — চলবে: Node.js

```
import { visibleRange } from '../../project/travel-search/src/virtual.mjs';

const range = visibleRange({
  scrollTop: 1_000, viewport: 600, rowHeight: 40, total: 500, overscan: 3,
});
console.log(range, range.end - range.start);
```

Output

```
{ start: 22, end: 43 } 21
```

প্রথম দৃশ্যমান সারি ১০০০ ÷ ৪০ = ২৫, শেষ (১০০০ + ৬০০) ÷ ৪০ = ৪০; Overscan ৩ দুই পাশে: ২২ থেকে ৪৩ (শেষটি বাদ), মোট ২১টি।

সমাধান ৫ — DOM Main Thread-এর Object এবং একসঙ্গে দুটি Thread থেকে বদলানো নিরাপদ নয়, তাই Worker DOM পায় না; সে Message পাঠায়, এবং Main Thread DOM বদলায়।

সমাধান ৬

long-measures.mjs · JavaScript — চলবে: Node.js

```
import { PerformanceObserver, performance } from 'node:perf_hooks';

const slow = [];
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    if (entry.duration > 5) {
      slow.push(entry.name);
    }
  }
});
observer.observe({ entryTypes: ['measure'] });

function work(name, ms) {
  performance.mark(name);
  const end = Date.now() + ms;
  while (Date.now() < end) {
    // নির্দিষ্ট সময় ব্যস্ত থাকা
  }
  performance.measure(name, name);
}
```

```
work('quick', 0);
work('report', 40);
setTimeout(() => {
  observer.disconnect();
  console.log(slow);
}, 0);
```

Output

```
[ 'report' ]
```

`quick` প্রায় শূন্য সময়, `report` অন্তত ৪০ ms; ২০ ms-এর সীমা থেকে দুটিই দূরে, তাই ফল স্থির। বাস্তবে এই ধরনের Observer ধীর কাজ Monitoring-এ পাঠায়।

সমাধান ৭

profile-two.mjs · JavaScript — চলবে: Node.js

```
import { Session } from 'node:inspector/promises';
import { topFunctions } from '../../project/travel-search/src/profile-report.mjs';

function heavy() {
  let total = 0;
  for (let i = 0; i < 3e7; i += 1) {
    total += Math.sqrt(i);
  }
  return total;
}

function light() {
  return Array.from({ length: 1_000 }, (_, i) => i).reduce((a, b) => a + b, 0);
}

const session = new Session();
session.connect();
await session.post('Profiler.enable');
await session.post('Profiler.start');
heavy();
light();
const { profile } = await session.post('Profiler.stop');
const [top] = topFunctions(profile, 1);
console.log(top.name.startsWith('heavy'));
```

Output

```
true
```

`heavy` কয়েক কোটি বার Loop চালায়, `light` এক হাজার; Profile-এ সবচেয়ে বেশি Sample `heavy`-তে—এটি সময়ের সীমা নয়, কাজের পরিমাণের বিশাল পার্থক্য, তাই ফল স্থির। নিজের যন্ত্রে `top.percent` ছাপিয়ে দেখুন।

সমাধান ৮ — Worker-এ প্রতিটি অনুরোধের `requestId` রাখুন এবং একটি `latest` Variable: নতুন Message এলে `latest = requestId`। খোঁজ যদি টুকরোয় চলে (প্রতি কয়েক হাজার Record-এর পর `await` একটি Task), প্রতিটি টুকরোর পর পরীক্ষা করুন `requestId === latest`; না হলে কাজ থামিয়ে কিছুই পাঠাবেন না। Project-এর মতো খোঁজ যদি এক টানে শেষ হয়, তাহলে বাতিল সম্ভব নয়, তাই Main Thread-এ পুরোনো উত্তর বাদ দেওয়াই (Project-এর `requestId` যাচাই) যথেষ্ট। আরও বড় কাজে `worker.terminate()` করে নতুন Worker তৈরি করাও একটি উপায়, যদিও তখন Index আবার বানাতে হয়।

সমাধান ৯

listener-leak.mjs · JavaScript — চলবে: Node.js

```
import { setFlagsFromString } from 'node:v8';
import { runInNewContext } from 'node:vm';

setFlagsFromString('--expose-gc');
const gc = runInNewContext('gc');
const heapMB = () => {
  gc();
  return process.memoryUsage().heapUsed / 1024 / 1024;
};

const bus = new EventTarget();
function openPanel(signal) {
  const bigState = new Array(20_000).fill('x');
  bus.addEventListener('refresh', () => bigState.length, { signal });
}

const start = heapMB();
for (let i = 0; i < 200; i += 1) {
  openPanel();
}
const leaked = heapMB() - start;

const controllers = [];
for (let i = 0; i < 200; i += 1) {
  const controller = new AbortController();
  openPanel(controller.signal);
  controllers.push(controller);
}
const beforeAbort = heapMB();
controllers.forEach((c) => c.abort());
await new Promise((resolve) => setTimeout(resolve, 0));
const freed = beforeAbort - heapMB();
```

```
console.log(`Signal ছাড়া: ২০ MB-র বেশি আটকে আছে: ${leaked > 20}`);
console.log(`abort()-এর পর ২০ MB-র বেশি মুক্ত: ${freed > 20}`);
```

Output

```
Signal ছাড়া: ২০ MB-র বেশি আটকে আছে: true
abort()-এর পর ২০ MB-র বেশি মুক্ত: true
```

প্রতিটি Listener-এর Closure ২০,০০০ ঘরের একটি Array ধরে রাখে, এবং Listener `bus`-এ যুক্ত থাকায় সেগুলো কখনো মুক্ত হয় না—২০০ বার Panel খোলা মানে প্রায় ৩০ MB। `signal`-সহ যুক্ত Listener `abort()`-এ সরে যায়, Closure পৌঁছানোর অযোগ্য হয়, এবং GC Memory ফেরত নেয়। একটি বাস্তব পর্যবেক্ষণ: এই বইয়ের পরীক্ষায় (Node.js 22.16.0) `abort()`-এর ঠিক পরে মাপলে Memory তখনো মুক্ত হয়নি; একটি Event Loop চক্র (`setTimeout(..., 0)`) পরে মুক্ত হয়েছে, তাই Code-এ সেই অপেক্ষা। আরেকটি সূত্র: প্রথম Loop চালানোর সময় Node.js নিজেই Terminal-এ সতর্ক করে—`MaxListenersExceededWarning: Possible EventTarget memory leak detected`—একই Event-এ দশটির বেশি Listener যুক্ত হলে। এই সতর্কবার্তা উপেক্ষা করবেন না; এটি প্রায়ই সত্যিকারের Leak।

সমাধান ১০

variable-rows.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createVariableList(heights) {
  const tops = [0];
  for (const h of heights) {
    tops.push(tops.at(-1) + h);
  }
  // যে সারির শুরু y-এর ঠিক আগে বা y-এ, তার Index (Binary Search)।
  const rowAt = (y) => {
    let low = 0;
    let high = heights.length - 1;
    while (low < high) {
      const middle = (low + high + 1) >>> 1;
      if (tops[middle] <= y) {
        low = middle;
      } else {
        high = middle - 1;
      }
    }
    return low;
  };
  return {
    totalHeight: tops.at(-1),
    range: (scrollTop, viewport) =>
      ({ start: rowAt(scrollTop), end: rowAt(scrollTop + viewport - 1) + 1 }),
    offsetOf: (index) => tops[index],
  };
```

```
}

const list = createVariableList([40, 80, 40, 120, 40, 40, 200, 40]);
const range = JSON.stringify(list.range(100, 150));
console.log(list.totalHeight, range, list.offsetOf(3));
```

Output

```
600 {"start":1,"end":4} 160
```

Prefix Sum (`tops`) একবার হিসাব হয় O(n)-এ, তারপর প্রতিটি Scroll-এ খোঁজা O(log n)। ১০০ থেকে ২৪৯ পিক্সেল দেখা যায়: সারি ১ (৪০–১১৯), ২ (১২০–১৫৯), ৩ (১৬০–২৭৯)। বাস্তবে উচ্চতা আগে জানা থাকে না; Library-গুলো আনুমানিক উচ্চতা দিয়ে শুরু করে এবং আঁকার পর মেপে সংশোধন করে।

সমাধান ১১ — Index তৈরির সময় `rating` -এ সাজানো একটি Record-এর তালিকা ও প্রতিটি Record-এর অবস্থান (`rankById`) রাখুন। খোঁজার পর ফলকে তুলনা-ভিত্তিক `sort` -এর বদলে `rankById` দিয়ে সাজালে তুলনাগুলো সস্তা হয়; আর ফল যদি পুরো তালিকার বড় অংশ হয়, সাজানো তালিকা থেকে `filter` করাই O(n)। Test: Project-এর Property Test-টি `sort: 'rating'` দিয়ে চালান—নতুন সংস্করণ ও `naiveSearch` হুবহু সমান হতে হবে, বিশেষ করে সমান Rating-এ `id` -এর ক্রম। প্রথমে মাপুন: ১২,০০০ Record-এ এই Optimization-এর প্রয়োজন আছে কি না, Profile বলবে।

সমাধান ১২ — Worker শুধু ID-র একটি `Int32Array` পাঠায় এবং তার `buffer` Transfer করে—Clone-এর খরচ প্রায় শূন্য। Main Thread-এর কাছে Record থাকতে হবে: হয় শুরুতে একবার Worker থেকে সব Record পাঠানো (একবারের খরচ), নয়তো দৃশ্যমান সারির Record আলাদা Message-এ চাওয়া। লাভ: প্রতিটি খোঁজে ছোট Message। ক্ষতি: Main Thread-এ তথ্যের দ্বিতীয় Copy (Memory), অথবা Scroll-এর সময় অতিরিক্ত Message ও অপেক্ষা। ১২,০০০ ছোট Record-এ Project-এর সরল উপায় যথেষ্ট; লাখো Record-এ ID পাঠানোর নকশা যুক্তিসংগত—সিদ্ধান্ত মেপে নিন।

২৩.১৪ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `performance.measure('search', 'a', 'b')` কী করে?

ক) একটি নামযুক্ত Entry তৈরি করে, যাতে `a` থেকে `b` চিহ্নের মধ্যের সময়

খ) Function দ্রুত করে

গ) CPU Profile শুরু করে

ঘ) `a` ও `b` Function দুটি চালায়

প্রশ্ন ২. CPU Profile-এ "Self Time" কী?

ক) Program-এর মোট সময়

খ) Function নিজের Code-এ কাটানো সময়, ডাকা Function বাদে

গ) GC-র সময়

ঘ) Network-এর সময়

প্রশ্ন ৩. অধ্যায়ের Profile-এ সরল খোঁজের প্রধান খরচ কোথায় ছিল?

ক) `sort`-এ

খ) Network-এ

গ) প্রতিটি প্রশ্নে সব Record-এর লেখা আবার শব্দে ভাগ করায়

ঘ) DOM-এ

প্রশ্ন ৪. Memory Leak প্রমাণের সময় Heap মাপার আগে কী করা উচিত?

ক) Program বন্ধ

খ) Browser Refresh

গ) কিছুই না

ঘ) Garbage Collection চালানো বা Heap Snapshot নেওয়া

প্রশ্ন ৫. কোন CSS পরিবর্তন সাধারণত Layout ট্রিগার করে না?

ক) `transform`

খ) `height`

গ) `width`

ঘ) `top`

প্রশ্ন ৬. Virtual Scrolling-এ Spacer Element-এর কাজ কী?

ক) সারি আঁকা

খ) সব সারির মোট উচ্চতা ধরে রেখে Scrollbar সঠিক রাখা

গ) Worker চালানো

ঘ) খোঁজ করা

প্রশ্ন ৭. Web Worker কী করতে পারে না?

ক) `postMessage`

খ) Module `import`

গ) সরাসরি DOM বদলানো

ঘ) ভারী হিসাব

প্রশ্ন ৮. `ArrayBuffer` Transfer করার পর মূল Buffer-এর `byteLength`?

ক) একই থাকে

খ) দ্বিগুণ

গ) Error

ঘ) ০

প্রশ্ন ৯. Async খোঁজে `requestId` কেন?

ক) পুরোনো ধীর উত্তর নতুন উত্তরকে মুছে না দিতে

খ) খোঁজ দ্রুত করতে

গ) Memory কমাতে

ঘ) Worker বন্ধ করতে

প্রশ্ন ১০. Index-এর মতো Optimization-এর পর প্রথম যাচাই কী?

ক) Coverage ১০০%

খ) ফল সরল সংস্করণের সঙ্গে হুবহু মেলে

গ) Code ছোট

ঘ) Benchmark-এ ১০ গুণ দ্রুত

২৩.১৫ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–ক · ২–খ · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ঘ · ৯–ক · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	ক	নামযুক্ত পরিমাপ, যা DevTools ও Observer দেখতে পায়।
২	খ	কোন লাইন ব্যয়বহুল, তা Self Time বলে।
৩	গ	`words` প্রায় দুই-তৃতীয়াংশ সময়; সমাধান Index।
৪	ঘ	নইলে অমুক্ত-কিন্তু-মৃত Object-ও গোনা হয়।
৫	ক	Composite ধাপেই সরানো যায়।
৬	খ	DOM-এ সারি কম, কিন্তু Scroll-এর আকার পূর্ণ তালিকার।
৭	গ	Worker-এ `document` নেই; Message দিয়ে Main Thread-কে জানায়।
৮	ঘ	মালিকানা হস্তান্তর; মূলটি Detached।
৯	ক	Race Condition প্রতিরোধ।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	খ	দ্রুত কিন্তু ভুল ফল অর্থহীন।

২৩.১৬ অধ্যায়ের সারসংক্ষেপ

- Profile আগে: User Timing বলে কতক্ষণ, CPU Profile বলে কোথায়; Self ও Total Time দুটিই দেখুন।
- `performance.mark` / `measure` নামযুক্ত পরিমাপ; `PerformanceObserver` সেগুলো আলাদা জায়গায় সংগ্রহ করে।
- Event Loop Delay-এর Histogram Server-এর আটকে থাকা ধরে; Memory মাপুন GC-র পরে।
- Rendering: JavaScript, Style, Layout, Paint, Composite; `transform` ও `opacity` সস্তা।
- Virtual Scrolling: Spacer, বিশুদ্ধ হিসাব, `translateY`; সীমাবদ্ধতা জেনে ব্যবহার।
- Web Worker Main Thread মুক্ত রাখে; Message Clone হয়, `ArrayBuffer` Transfer করা যায়।
- Async খোঁজে `requestId` দিয়ে Race Condition রোধ।
- প্রতিটি Optimization-এর আগে-পরে সঠিকতার Test; সময় কখনো Test-এর শর্ত নয়।

পরের অধ্যায়ে JavaScript থেকে TypeScript: Static Type কীভাবে অনেক ত্রুটি Code চালানোর আগেই ধরে, এবং কোথায় তার সীমা।

২৩.১৭ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
User Timing	`mark` ও `measure` দিয়ে নামযুক্ত পরিমাপ।
CPU Profile	Sampling দিয়ে কোন Function কত সময় চলেছে তার রেকর্ড।
Self Time	Function নিজের Code-এ কাটানো সময়।
Total Time	ডাকা Function-সহ মোট সময়।
Event Loop Delay	Event Loop-এর একটি চক্র কত দেরিতে চলেছে।
Heap Snapshot	একটি মুহূর্তে Memory-র সব Object-এর চিত্র।
Layout	Element-এর আকার ও অবস্থানের হিসাব।
Composite	আঁকা স্তরগুলো জোড়া লাগানো।
Virtual Scrolling	শুধু দৃশ্যমান সারি আঁকা।
Overscan	দৃশ্যমান অংশের বাইরে আঁকা বাড়তি সারি।

পরিভাষা	অর্থ
Web Worker	আলাদা Thread-এ চলা Script।
Transferable	Copy ছাড়াই মালিকানা হস্তান্তরযোগ্য Object।
Inverted Index	শব্দ থেকে সেই শব্দযুক্ত Record-এর তালিকা।

২৩.১৮ আরও পড়ুন

- **MDN — User Timing:** `https://developer.mozilla.org/en-US/docs/Web/API/Performance_API/User_timing` — `mark` ও `measure`।
- **Node.js — Performance measurement APIs:** `https://nodejs.org/api/perf_hooks.html` — `PerformanceObserver`, `monitorEventLoopDelay`।
- **web.dev — Rendering performance:** `https://web.dev/articles/rendering-performance` — Pipeline ও `transform`।
- **MDN — Using Web Workers:** `https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers` — Worker ও Transferable।
- **Chrome DevTools — Fix memory problems:** `https://developer.chrome.com/docs/devtools/memory-problems` — Heap Snapshot।